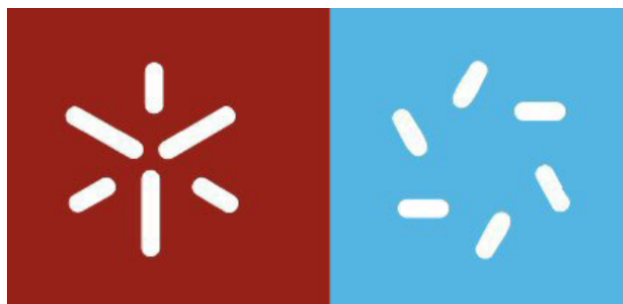




UNIVERSIDADE DO MINHO
LICENCIATURA EM CIÊNCIAS DA COMPUTAÇÃO

Sistemas Operativos

Trabalho Prático

Grupo 22

Rita Machado
(A102508)

Catarina Rodrigues
(A111491)

Francisca Cardoso
(A112105)

Contents

1. Introdução	3
2. Funcionalidades Básicas e Avançadas	4
2.1 Funcionalidades Básicas	4
2.1.1 Execute (-e)	4
2.1.2 Status (-c)	4
2.1.3 Shutdown (-s)	4
2.2 Funcionalidades Avançadas	4
3. Implementação	5
3.1 Mecanismos de Comunicação e Sincronização	5
3.2 Cliente (runner.c)	6
3.2.1 Funções auxiliares	6
3.3 Servidor (controller.c)	7
3.3.1 Funções auxiliares	7
4. Conclusão	8

1. Introdução

No âmbito da unidade curricular de Sistemas Operativos, foi proposta a implementação de um sistema de orquestração de um ambiente multi-runner, constituído por dois programas distintos: um cliente (runner) e um servidor central (controller).

O principal objetivo deste projeto é permitir que múltiplos utilizadores submetam comandos de forma concorrente através do runner, sendo o controller responsável por escalonar, limitar as execuções em paralelo e registar o histórico de tarefas.

2. Funcionalidades Básicas e Avançadas

2.1 Funcionalidades Básicas

2.1.1 Execute (-e)

Ao submeter um comando do tipo `./bin/runner -e`, seguido de um identificador numérico de utilizador e de uma string constituída pelo comando bash a executar (e respetivos argumentos, caso existam), irá ser escrito no *output* do cliente o momento exato de submissão da tarefa, o aviso de início de execução (após autorização do orquestrador) e a confirmação do seu término. Paralelamente, o orquestrador calcula o tempo total da operação em segundos (incluindo o compasso de espera na fila) e anexa esse registo de forma persistente no ficheiro de log.

2.1.2 Status (-c)

Se submeter um comando do tipo `./bin/runner -c`, sem quaisquer argumentos adicionais, irá ser escrito no output do cliente uma lista estruturada em duas secções distintas. Esta lista exhibe em tempo real o identificador do utilizador (`user-id`) e o identificador do processo (`command-id`) de todas as tarefas que se encontrem atualmente em processamento ativo ou a aguardar vaga na fila de espera do servidor.

2.1.3 Shutdown (-s)

Se submeter um comando do tipo `./bin/runner -s`, sem quaisquer argumentos adicionais, o cliente enviará uma notificação de encerramento ao orquestrador e irá ser escrito no *output* uma mensagem a indicar que se encontra a aguardar o fim das operações do servidor. O servidor passará a rejeitar ativamente novos pedidos e, assim que todas as tarefas em execução terminarem, irá ser escrito no output do cliente a confirmação final de que o orquestrador encerrou de forma segura.

2.2 Funcionalidades Avançadas

- **Processamento Paralelo:** O orquestrador atua como um gestor de concorrência que limita o número de tarefas a executar em simultâneo com base num argumento fornecido no arranque (`max_parallel`). Tarefas excedentes são retidas numa fila de espera.
- **Políticas de Escalonamento:** A política implementada foi a FIFO (First-In, First-Out), garantindo que as tarefas na fila de espera são atendidas pela sua ordem de chegada assim que é libertada uma vaga no servidor.

3. Implementação

3.1 Mecanismos de Comunicação e Sincronização

A arquitetura do sistema baseia-se num modelo cliente-servidor distribuído. Dado que os processos *runner* e *controller* não possuem parentesco nem partilham espaço de endereçamento, a comunicação foi estabelecida através de *Named Pipes* (FIFOs).

- **Canal de Pedidos:** Foi implementado um FIFO global (*fifo_controller*) para a receção de mensagens. Para garantir a integridade dos dados na escrita concorrente de múltiplos clientes, as mensagens são encapsuladas numa estrutura *Message* de tamanho fixo. Esta abordagem assegura que as operações de escrita no pipe sejam atómicas, evitando o entrelaçamento de dados.
- **Canais de Resposta:** Cada instância do *runner* cria dinamicamente um FIFO privado (*fifo_runner_<pid>*) antes de submeter o pedido. Esta estratégia é fundamental para o isolamento de processos: garante que o orquestrador direcione autorizações, listas de estado ou notificações de erro exclusivamente ao cliente que as solicitou, sem interferência de outros processos concorrentes.
- **Prevenção de *Busy Waiting*:** No orquestrador, o descritor do FIFO principal é aberto com a *flag O_RDWR*. Esta decisão impede que a chamada *read()* retorne 0 quando todos os clientes se desconectam, forçando o processo a um bloqueio passivo pelo Kernel enquanto aguarda novos pedidos, otimizando assim o consumo de CPU.

3.2 Cliente (runner.c)

Adotamos uma implementação de uma estrutura de código que permite ler e interpretar os argumentos escritos na linha de comandos. Consoante o argumento, o programa segue fluxos distintos:

- "execute" (-e):
Ao submeter este comando, o cliente inicializa uma estrutura Message contendo o *PID*, o ID do utilizador e o comando a executar. É criado um pipe com nome privado (fifo_runner_<pid>) para receber a resposta do servidor. Após enviar o pedido ao orquestrador, o cliente bloqueia à espera de autorização. Recebido o sinal "OK", é criado um processo filho através de *fork()*. O processo filho executa o comando através da função *execvp()*, enquanto o processo pai aguarda a sua finalização via *waitpid()*. No final, o cliente envia uma notificação de término ao servidor para que a vaga seja libertada e o tempo seja registado.
- "status" (-c):
O cliente envia uma mensagem ao orquestrador que identifica a operação de consulta. De seguida, abre o seu pipe privado para leitura e recebe uma string formatada que contém todos os processos em execução e agendados, imprimindo-a diretamente no standard output.
- "shutdown" (-s):
É enviada uma flag específica que sinaliza ao servidor a intenção de encerrar o serviço. O cliente aguarda no seu pipe privado a confirmação de que o orquestrador terminou todas as tarefas pendentes e encerrou com sucesso.

3.2.1 Funções auxiliares

- "write_to_controller": Abre o pipe central fifo_controller exclusivamente para escrita e envia a estrutura Message para o servidor.
- Parser: Esta lógica percorre a string do comando e separa-a em argumentos individuais num vetor, tratando corretamente espaços e aspas para garantir que o *execvp()* receba o formato correto.

3.3 Servidor (controller.c)

Para o servidor, desenvolvemos uma função principal que começa por criar o pipe de sistema `fifo_controller`. O servidor entra num ciclo contínuo de leitura onde, consoante a operação recebida, gere a concorrência e o histórico:

- Execução de Tarefa (`operation == 'e'`):
 - Início (*status* 0): Ao receber um novo programa, o servidor regista o instante de chegada através de `gettimeofday()`. Se o número de processos ativos for inferior ao limite permitido, envia autorização ao cliente e adiciona o processo à lista de processos ativos. Caso contrário, a tarefa é colocada numa fila de espera FIFO.
 - Fim (*status* 1): Quando recebe a notificação de término, o servidor calcula o tempo de execução total, remove o processo da lista de ativos e regista a informação no ficheiro de log. De seguida, verifica se existem processos na fila de espera para os iniciar imediatamente.
- Consulta de Estado (`operation == 'c'`): O servidor itera sobre as listas de processos ativos e agendados, concatenando os dados num *buffer* que é enviado de volta para o cliente através do seu pipe específico.
- Encerramento (`operation == 's'`): O servidor ativa uma flag de encerramento. A partir desse momento, novos pedidos são rejeitados. O ciclo principal termina apenas quando todos os processos em execução terminarem e a fila de espera estiver vazia.

3.3.1 Funções auxiliares

- "log_task": Abre o ficheiro `log_tasks.txt` com a flag `O_APPEND` e escreve o histórico da tarefa, incluindo ID do utilizador, PID e a duração precisa da execução.
- "send_authorization": Identifica o cliente pelo PID e envia o sinal de controlo "OK" através do pipe privado do respetivo *runner*.
- "remove_from_executing": Percorre a lista de tarefas ativas e remove o processo terminado, reorganizando o *array* para evitar lacunas na gestão de concorrência.
- "pop_from_queue": Retira o primeiro elemento da fila de agendamento (política FIFO) e desloca os restantes elementos, preparando a próxima tarefa para execução.

4. Conclusão

Como é possível verificar através da implementação e dos testes efetuados, o nosso trabalho executa corretamente as funcionalidades implementadas, nomeadamente as funcionalidades básicas e os processamento paralelo. Logo, acreditamos que o objetivo final foi bem conseguido, resultando num sistema de orquestração de processos estável e funcional.

Apesar de alguns desafios, tais como a implementação de um analisador sintático manual para processar os comandos sem recorrer a interpretadores externos, e a gestão rigorosa da comunicação bidirecional através de pipes nominais (FIFOs), a consulta às resoluções dos guiões práticos apresentados pela equipa docente foi uma peça fundamental que nos ajudou a superar estas barreiras.

Este trabalho permitiu-nos consolidar o que fomos aprendendo ao longo deste semestre na unidade curricular de Sistemas Operativos de uma maneira prática. A aplicação direta de conceitos como a criação de processos (*fork*), a execução de programas (*execvp*), a sincronização (*waitpid*) e a utilização estrita de chamadas de sistema para manipulação de ficheiros e comunicação IPC revelou-se uma experiência enriquecedora que solidificou a nossa compreensão sobre o funcionamento interno do sistema operativo.